《通信与网络》
Communications & Networks

TCP and Transmission Control

Referred Book: Computer Networks: A Systems Approach, (5th edition), by Peterson and Davie: 5 End-to-End Protocols; 6 Congestion Control and Resource Allocation

Thanks to some coursework material and slice from Princeton, MIT and Berkley.

1

Outline

- The Transport Layer
- The UDP Protocol
- The TCP Protocol
  - TCP Segments
  - TCP Sequence Numbers
  - TCP Connection setup
  - Timeouts and Retransmission
  - TCP Sliding Window
  - Congestion Control and Avoidance

2

2

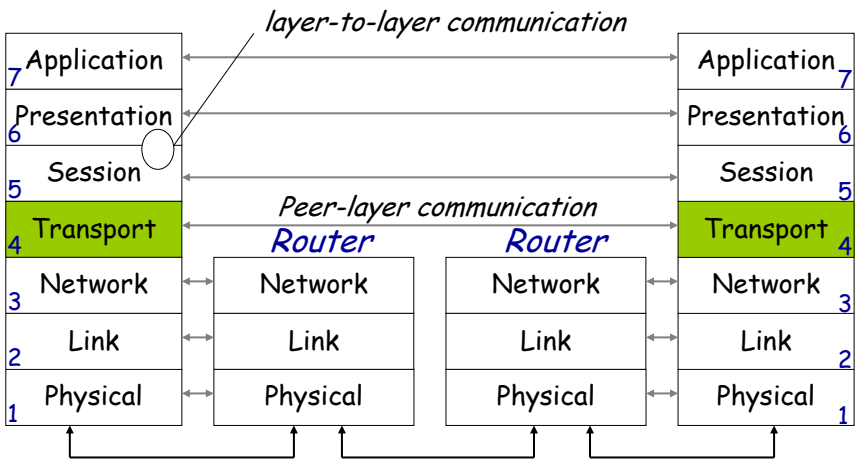
The Transport Layer

- What is the transport layer for?
  - Link & network layer, and why transport layer?
  - Many medias, different quality
    - Wireless, radio,
    - Wired, coaxial cable, twisted-pair cables, Ethernet cable, power cable
- What characteristics might it have?
  - Reliable delivery
  - Flow control
  - Congestion control

3

3

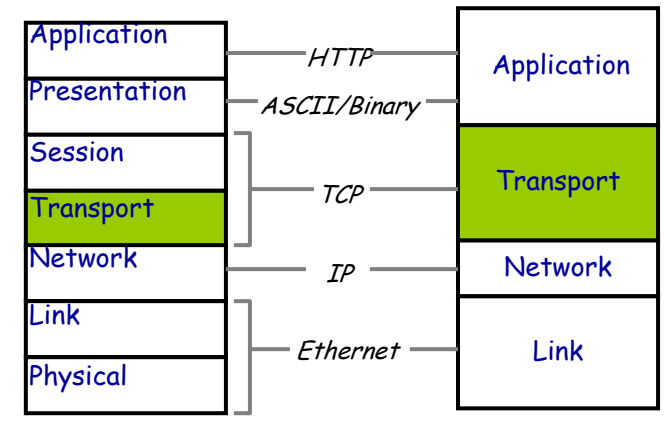
Layering: The OSI Model



5

5

Layering: Internet Example



6

6

IP Protocol Stack: Key Abstractions

- **Transport layer**
  - Provide applications with good abstractions
  - Without support or feedback from the network

7

7

Transport Protocols

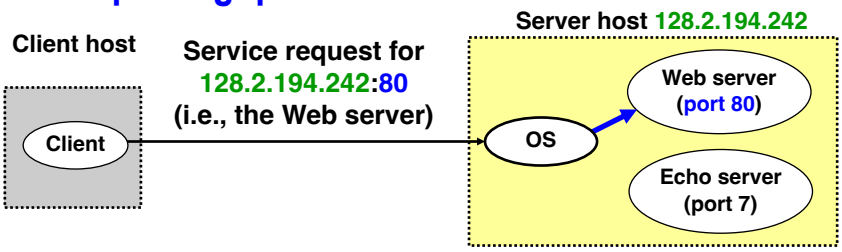
- Logical communication between processes
  - Sender divides a message into segments
  - Receiver reassembles segments into message
- Transport services
  - (De)multiplexing packets
  - Detecting corrupted data
  - Optionally: reliable delivery, flow control, …

8

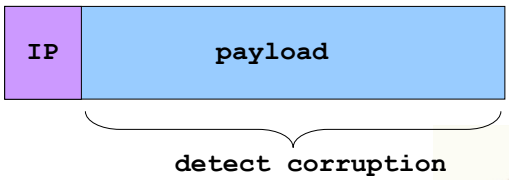
8

Two Basic Transport Features

- Demultiplexing: port numbers



- Error detection: checksums



9

9

UDP: User Datagram Protocol

- Fine-grain control
  - UDP sends as soon as the application writes
- No connection set-up delay
  - UDP sends without establishing a connection
- No connection state
  - No buffers, parameters, sequence #s, etc.
- Small header overhead
  - UDP header is only eight-bytes

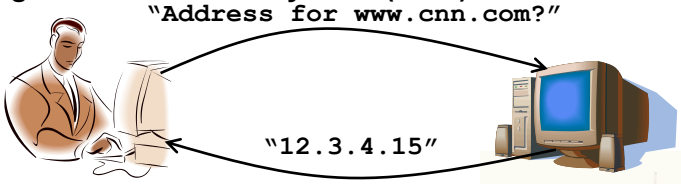
SRC port	DST port
checksum	length
DATA	

11

11

Popular Applications That Use UDP

- Multimedia streaming
  - Retransmitting packets is not always worthwhile
  - E.g., phone calls, video conferencing, gaming, IPTV
- Simple query-response protocols
  - Overhead of connection establishment is overkill
  - E.g., Domain Name System (DNS), DHCP, etc.



12

12

TRANSPORT LAYER: FROM UDP TO TCP

13

13

Transmission Control Protocol (TCP)

- **Stream-of-bytes service**
  - Sends and receives a stream of bytes
- **Reliable, in-order delivery**
  - Corruption: checksums
  - Detect loss/reordering: sequence numbers
  - Reliable delivery: acknowledgments and retransmissions
- **Connection oriented**
  - Explicit set-up and tear-down of TCP connection
- **Flow control**
  - Prevent overflow of the receiver's buffer space
- **Congestion control**
  - Adapt to network congestion for the greater good

14

14

BREAKING A STREAM OF BYTES INTO TCP SEGMENTS

15

15

TCP "Stream of Bytes" Service

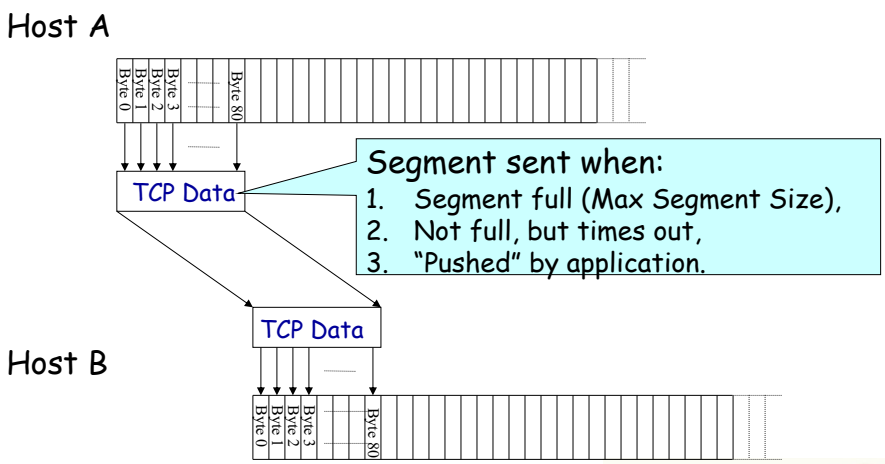
Host A

Host B

16

16

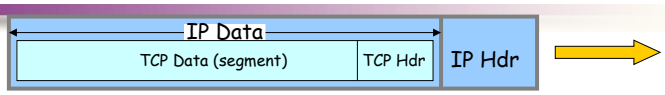
…Emulated Using TCP "Segments"



17

17

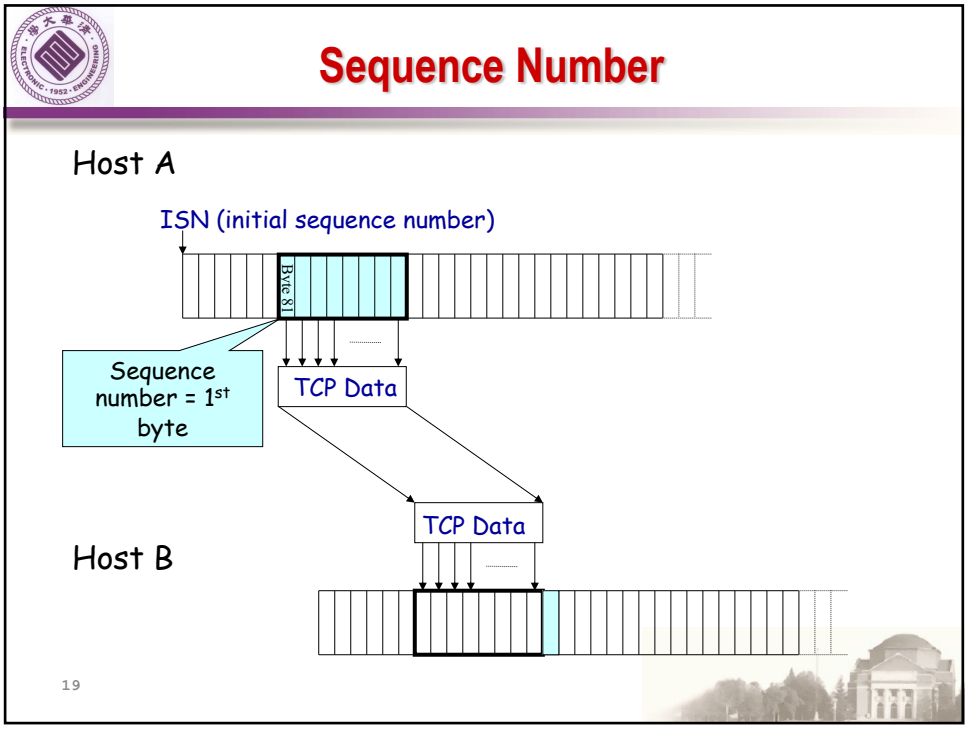
TCP Segment



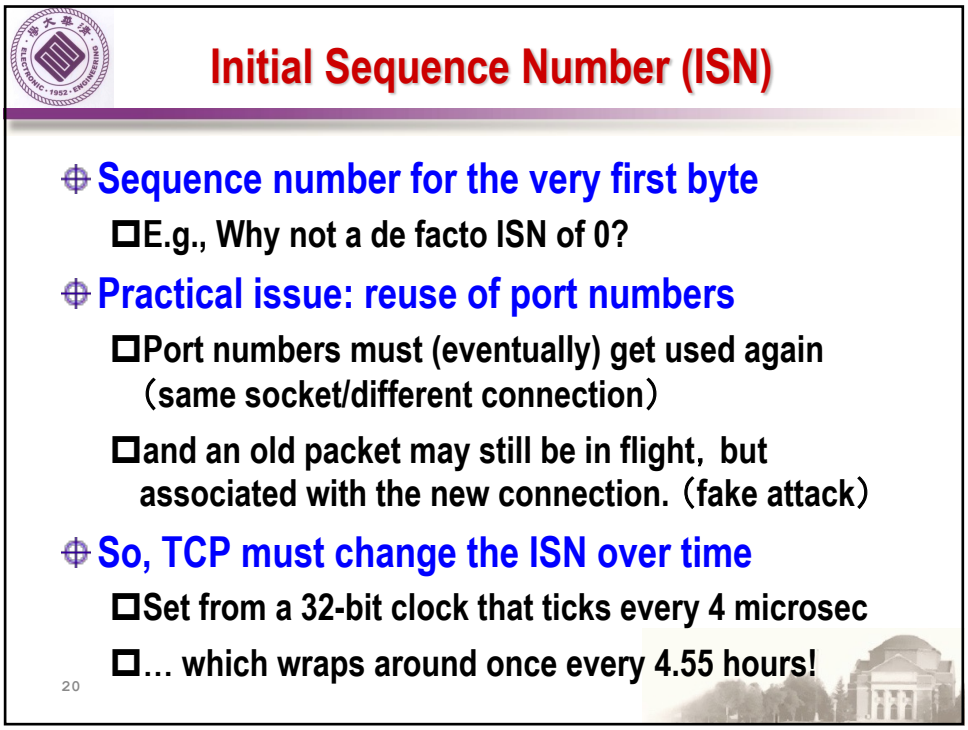
- **IP packet**
  - No bigger than Maximum Transmission Unit (MTU)
  - E.g., 1500 bytes on Ethernet link, 4352B@FDDI、
  - Maximum IP packet=65535 Bytes
- **TCP packet**
  - IP packet with a TCP header and data inside
  - TCP header is typically 20 bytes long
- **TCP segment**
  - No more than Maximum Segment Size (MSS) bytes
  - E.g., up to 1460 consecutive bytes from the stream

18

18


Sequence Number
Host A
ISN (initial sequence number)
Byte 81
Sequence number = 1st byte
TCP Data
TCP Data
Host B
19

19

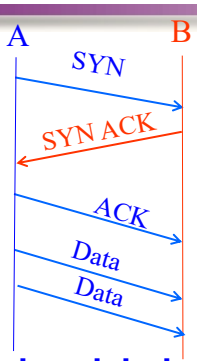

Initial Sequence Number (ISN)
Sequence number for the very first byte
E.g., Why not a de facto ISN of 0?
Practical issue: reuse of port numbers
Port numbers must (eventually) get used again (same socket/different connection)
and an old packet may still be in flight, but associated with the new connection. (fake attack)
So, TCP must change the ISN over time
Set from a 32-bit clock that ticks every 4 microsec
… which wraps around once every 4.55 hours!
20

20

STARTING AND ENDING A CONNECTION: TCP HANDSHAKES

21

Establishing a TCP Connection



Each host tells its ISN to the other host.

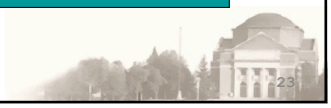
- Three-way handshake to establish connection
  - Host A sends a SYN (open) to the host B
  - Host B returns a SYN acknowledgment (SYN ACK)
  - Host A sends an ACK to acknowledge the SYN ACK

22

TCP Header

Flags: SYN
FIN
RST
PSH
URG
ACK

Source port			Destination port
Sequence number			
Acknowledgment			
HdrLen	0	**Flags**	Advertised window
Checksum			Urgent pointer
Options (variable)			
Data			



23

Step 1: A's Initial SYN Packet

Flags: SYN
FIN
RST
PSH
URG
ACK

A's port			B's port
A's Initial Sequence Number			
Acknowledgment			
20	0	Flags	Advertised window
Checksum			Urgent pointer
Options (variable)			

A tells B it wants to open a connection…

24

Step 2: B's SYN-ACK Packet

Flags: SYN
FIN
RST
PSH
URG
ACK

B's port			A's port
B's Initial Sequence Number			
A's ISN plus 1			
20	0	Flags	Advertised window
Checksum			Urgent pointer
Options (variable)			

B tells A it accepts, and is ready to hear the next byte…
… upon receiving this packet, A can start sending data

25

25

Step 3: A's ACK of the SYN-ACK

Flags: SYN
FIN
RST
PSH
URG
ACK

A's port			B's port
Sequence number			
B's ISN plus 1			
20	0	Flags	Advertised window
Checksum			Urgent pointer
Options (variable)			

A tells B it is okay to start sending
… upon receiving this packet, B can start sending data

26

26

What if the SYN Packet Gets Lost?

- Suppose the SYN packet gets lost
  - Packet is lost inside the network, or
  - Server rejects the packet (e.g., listen queue is full)
- Eventually, no SYN-ACK arrives
  - Sender sets a timer and wait for the SYN-ACK
  - … and retransmits the SYN if needed
- How should the TCP sender set the timer?
  - Sender has no idea how far away the receiver is
  - Some TCPs use a default of 3 or 6 seconds

27

27

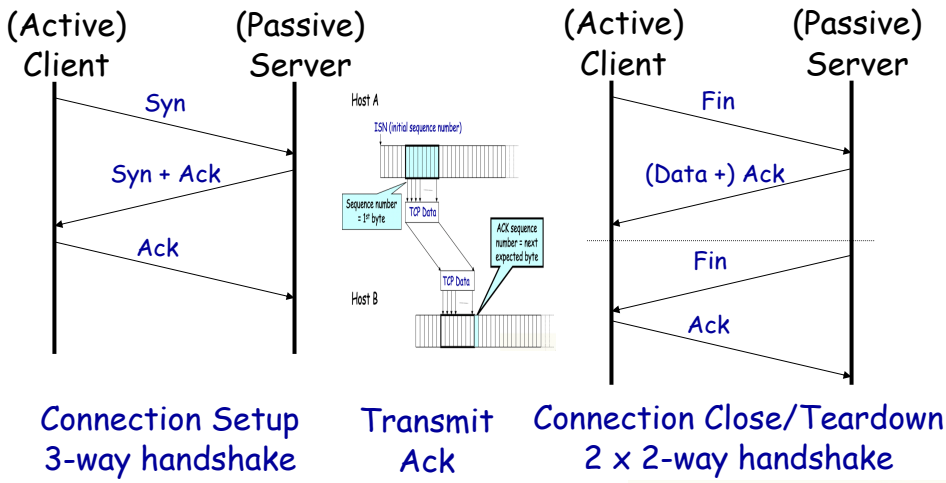
SYN Loss and Web Downloads

- User clicks on a hypertext link
  - Browser creates a socket and does a "connect"
  - The "connect" triggers the OS to transmit a SYN
- If the SYN is lost…
  - The 3-6 seconds of delay is very long
  - The impatient user may click "reload"
- User triggers an "abort" of the "connect"
  - Browser "connects" on a new socket
  - Essentially, forces a fast send of a new SYN!

28

28

TCP： Connect, Transmit, Close Connect



Connection Setup
3-way handshake

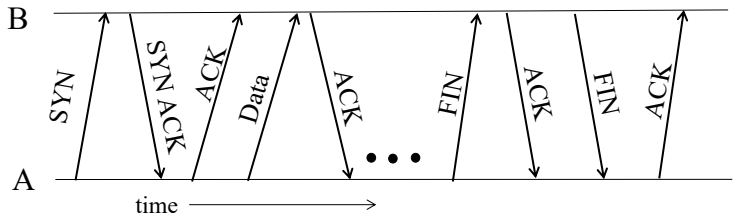
Transmit
Ack

Connection Close/Teardown
2 x 2-way handshake

29

29

Tearing Down the Connection



- Closing (each end of) the connection
  - Finish (FIN) to close and receive remaining bytes
  - And other host sends a FIN ACK to acknowledge
  - Reset (RST) to close and not receive remaining bytes

30

30

Sending/Receiving the FIN Packet

- **Sending a FIN: close()**
  - Process is done sending data via the socket
  - Process invokes "close()" to close the socket
  - Once TCP has sent all the outstanding bytes…
  - … then TCP sends a FIN
- **Receiving a FIN: EOF**
  - Process is reading data from the socket
  - Eventually, the attempt to read returns an EOF

RELIABLE DELIVERY ON A LOSSY CHANNEL WITH BIT ERRORS

Challenges of Reliable Data Transfer

- Over a perfectly reliable channel
  - Easy: sender sends, and receiver receives
- Over a channel with bit errors
  - Receiver detects errors and requests retransmission
- Over a lossy channel with bit errors
  - Some data are missing, and others corrupted
  - Receiver cannot always detect loss
- Over a channel that may reorder packets
  - Receiver cannot distinguish loss from out-of-order

33

33

An Analogy

- Alice and Bob are talking
  - What if Alice couldn't understand Bob?
  - Bob asks Alice to repeat what she said
- What if Bob hasn't heard Alice for a while?
  - Is Alice just being quiet? Has she lost reception?
  - How long should Bob just keep on talking?
  - Maybe Alice should periodically say "uh huh"
  - … or Bob should ask "Can you hear me now?" ☺

34

34

Take-Aways from the Example

- Acknowledgments from receiver
  - Positive: "okay" or "uh huh" or "ACK"
  - Negative: "please repeat that" or "NACK"
- Retransmission by the sender
  - After *not* receiving an "ACK"
  - After receiving a "NACK"
- Timeout by the sender ("stop and wait")
  - Don't wait forever without some acknowledgment

35

35

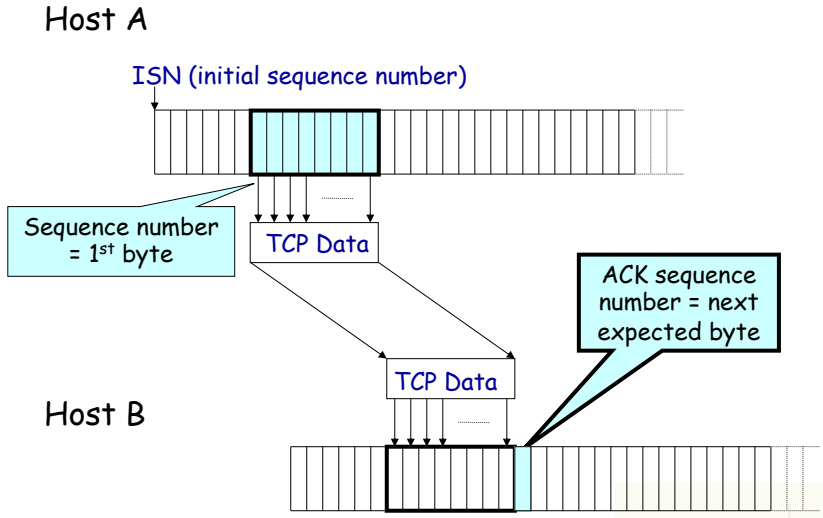
TCP Support for Reliable Delivery

- Detect bit errors: checksum
  - Used to detect corrupted data at the receiver
  - …leading the receiver to drop the packet
- Detect missing data: sequence number
  - Used to detect a gap in the stream of bytes
  - ... and for putting the data back in order
- Recover from lost data: retransmission
  - Sender retransmits lost or corrupted data
  - Two main ways to detect lost packets

36

36

TCP Acknowledgments

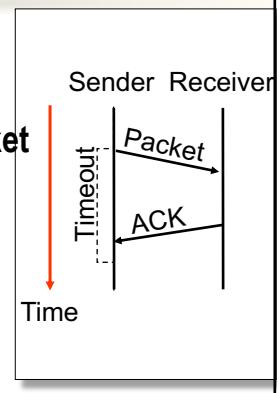


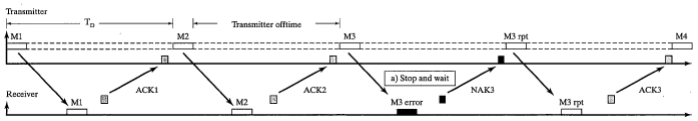
37

37

Automatic Repeat reQuest (ARQ)

- **ACK and timeouts**
  - Receiver sends ACK when receives packet
  - Sender waits for ACK and times out
- **Simplest ARQ protocol**
  - Stop and wait
  - Send a packet, stop and wait until ACK arrives



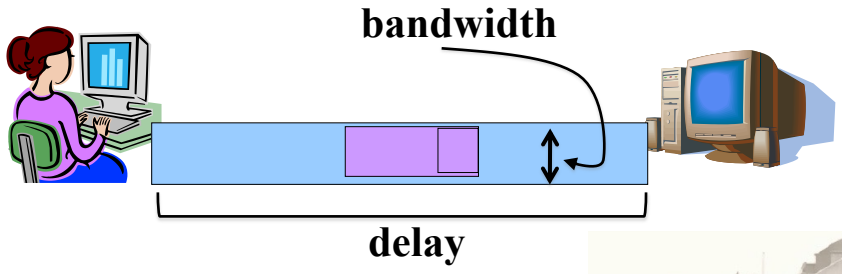


38

38

Stop-and-wait is inefficient

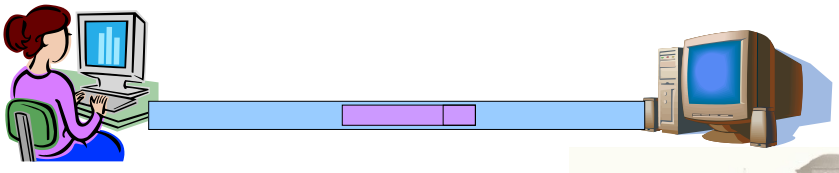
- Only one TCP segment is "in flight" at a time
- Especially bad for high "delay-bandwidth product"



39

Numerical Example

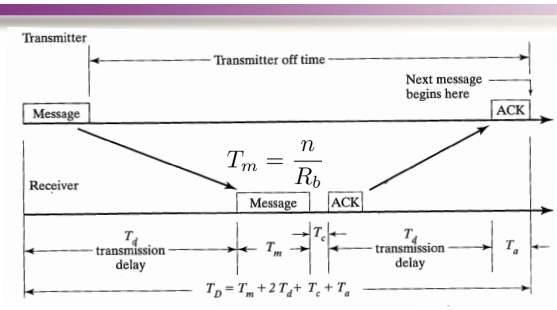
- 1.5 Mbps link with 45 msec round-trip time (RTT)
  - Delay-bandwidth product is 67.5 Kbits (or 8 KBytes)
- Sender can send at most one packet per RTT
  - Assuming a segment size of 1 KB (8 Kbits)
  - 8 Kbits/segment at 45 msec/segment ➔ 182 Kbps
  - That's just *one-eighth* of the 1.5 Mbps link capacity



40

40

Stop and wait



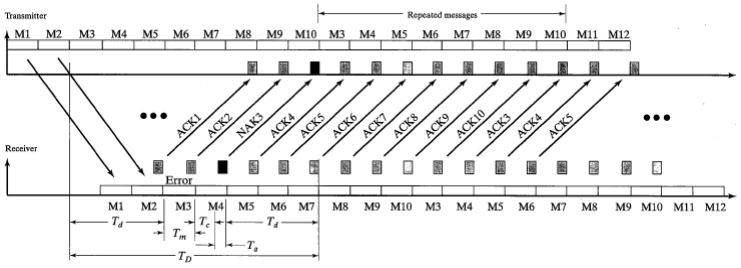
$$T_D = T_m + 2T_d + T_c + T_a$$

$$\widehat{R_b} = R_b \frac{T_m}{T_D}$$

$$\widehat{R_b} = R_b \frac{T_m}{\sum_i T_{D-i}}$$

41

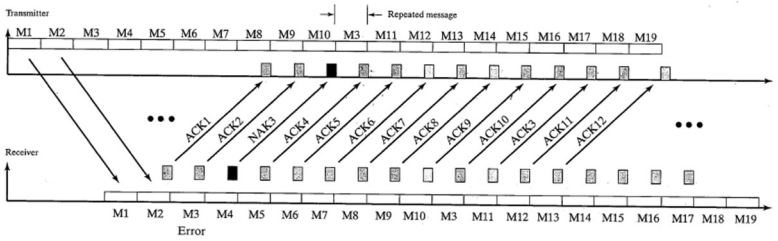
Go-Back-N ARQ



TCP's Retransmission policy is "Go Back N".

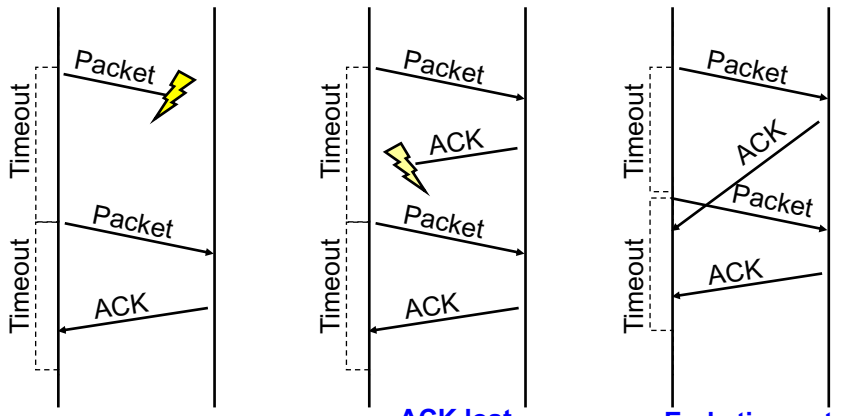
42

Selective Repeat ARQ



43

Reasons for Retransmission



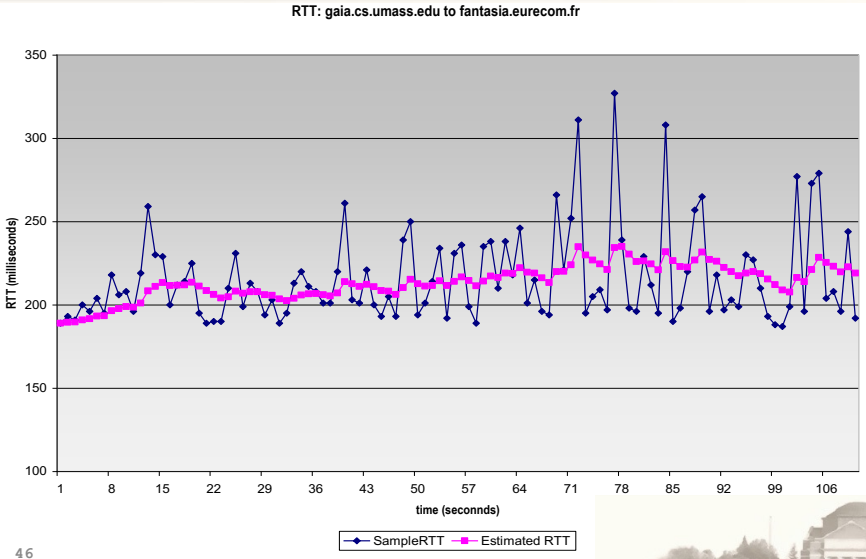
44

How Long Should Sender Wait?

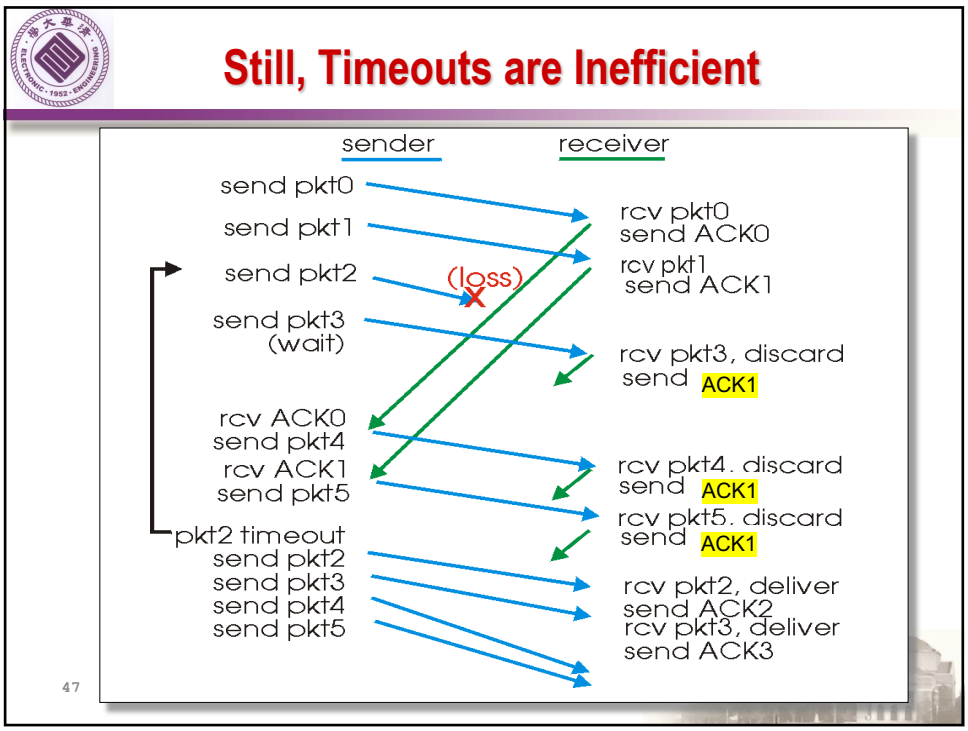
- Sender sets a timeout to wait for an ACK
  - Too short: wasted retransmissions
  - Too long: excessive delays when packet lost
- TCP sets timeout as a function of the RTT
  - Expect ACK to arrive after an "round-trip time"
- But, how does the sender know the RTT?
  - Running average of delay to receive an ACK

45

Example RTT Estimation



46



47

Fast Retransmission

- When packet n is lost…
  - … packets n+1, n+2, and so on may get through
- Exploit the ACKs of these packets
  - ACK says receiver is still awaiting nth packet
  - Duplicate ACKs suggest later packets arrived
  - Sender uses “duplicate ACKs” as a hint
- Fast retransmission
  - Retransmit after “triple duplicate ACK”

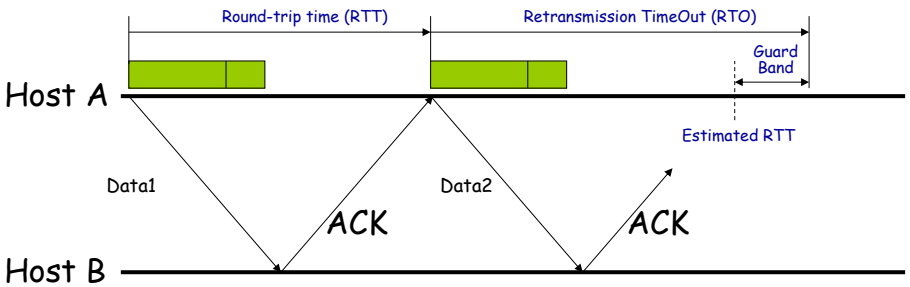
48

48

Effectiveness of Fast Retransmit

- When does Fast Retransmit work best?
  - High likelihood of many packets in flight
  - Long data transfers, large window size, …
- Implications for Web traffic
  - Most Web transfers are short (e.g., 10 packets)
    - So, often there aren't many packets in flight
  - Making fast retransmit is less likely to "kick in"
    - Forcing users to click "reload" more often… ☺

TCP: Retransmission and Timeouts



TCP uses an adaptive retransmission timeout value:
Congestion, Changes in Routing } RTT changes frequently

RTO: Retransmission Time Out

Picking the RTO is important:

- Pick a values that's too big and it will wait too long to retransmit a packet,
- Pick a value too small, and it will unnecessarily retransmit packets.

The original algorithm for picking RTO:

1. $\text{EstimatedRTT}_k = (1 - \alpha)\ \text{EstimatedRTT}_{k-1} + \alpha\ \text{SampleRTT}$
2. $\text{RTO} = 2 * \text{EstimatedRTT}$

Determined empirically

empirical value is 12.5%

Characteristics of the original algorithm:

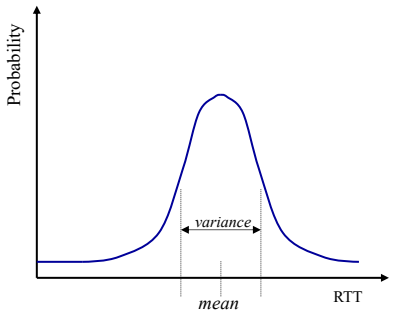
- Variance is assumed to be fixed.
- But in practice, variance increases as congestion increases.

51

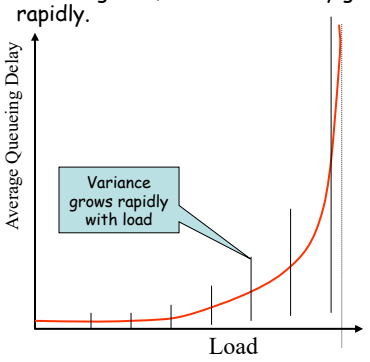
51

TCP: Retransmission and Timeouts

- There will be some (unknown) distribution of RTTs.
- We are trying to estimate an RTO to minimize the probability of a false timeout.



- Router queues grow when there is more traffic, until they become unstable.
- As load grows, variance of delay grows rapidly.



52

52

TCP: Retransmission and Timeouts

Newer Algorithm includes estimate of variance in RTT:

- Difference = SampleRTT - EstimatedRTT
- $\text{EstimatedRTT}_k = \text{EstimatedRTT}_{k-1} + (\delta * \text{Difference})$
- Deviation = Deviation + δ*(|Difference| - Deviation)

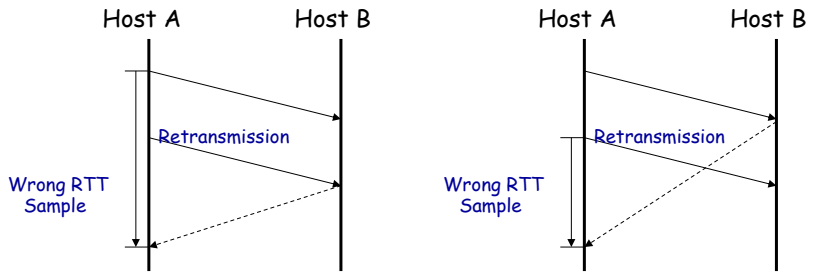
- RTO = μ * EstimatedRTT + ϕ * Deviation

  $\mu \approx 1$

  $\phi \approx 4$

TCP: Retransmission and Timeouts

Karn's Algorithm



Problem:
How can we estimate RTT when packets are retransmitted?
Solution:
On retransmission, don't update estimated RTT (and double RTO).